

LSV PA1 Report

Exercises 2–3

1 Exercise 2: Using ABC

2(a) Two-bit multiplier BLIF

File: `mul.blif`. Inputs $A = (a_1a_0)$, $B = (b_1b_0)$; outputs $Y = (y_3y_2y_1y_0) = A \cdot B$. Partial products $p_0=a_0b_0$, $p_1=a_0b_1$, $p_2=a_1b_0$, $p_3=a_1b_1$, then $y_0=p_0$, $y_1=p_1 \oplus p_2$, $y_2=p_3 \oplus (p_1p_2)$, $y_3=p_3(p_1p_2)$.

2(b) ABC session on `mul.blif`

Commands: `read` $\rightarrow$ `print_stats` $\rightarrow$ `show` $\rightarrow$ `strash` $\rightarrow$ `show` $\rightarrow$ `collapse` $\rightarrow$ `show bdd -g`.

Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 9 logic nodes and 0 latches.

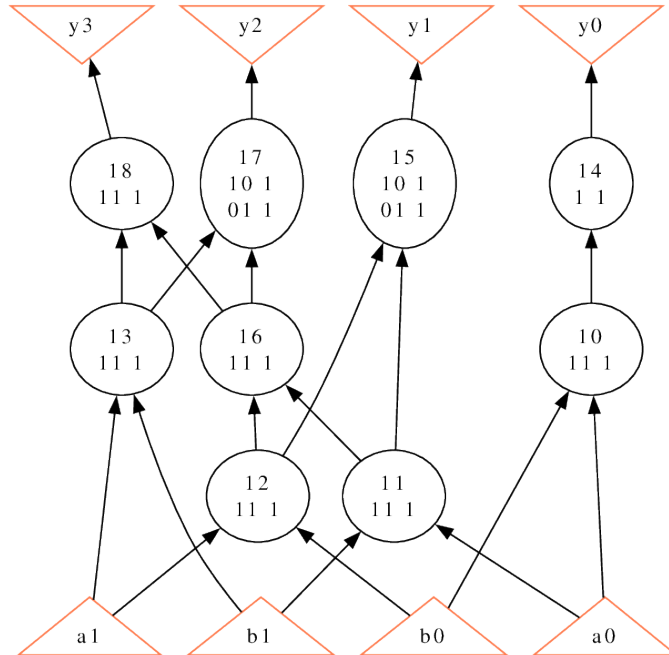


Figure 1: Logic network after `read` (`show`).

Network structure visualized by ABC
 Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 12 logic nodes and 0 latches.

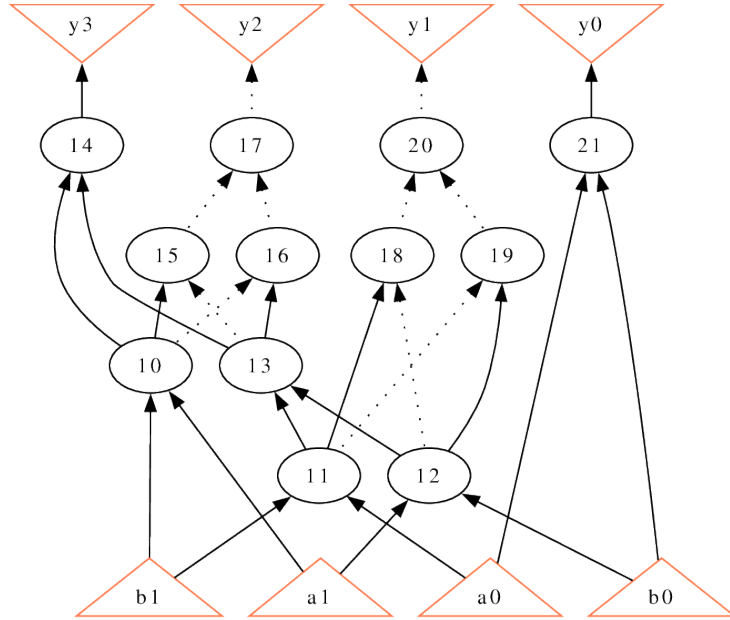


Figure 2: AIG after `strash (show)`.

Network structure visualized by ABC
 Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 4 logic nodes and 0 latches.

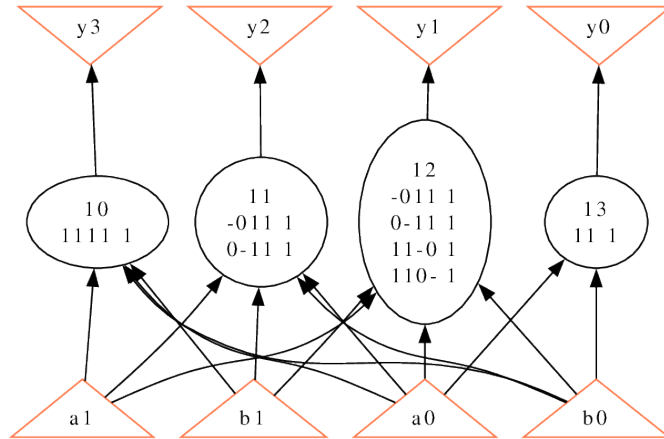


Figure 3: Collapsed BDDs after `collapse (show bdd -g)`.

print_stats after read: $i/o=4/4$, $nd=9$, $lev=3$. After strash: $and=12$, $lev=4$.

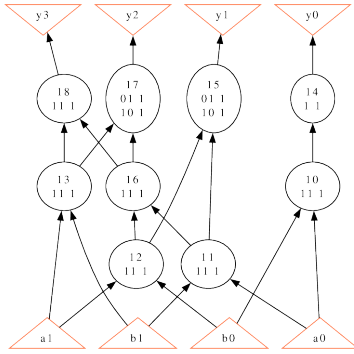
2 Exercise 3: Boolean Function Representations

3(a) Comparisons

AIG (aig) vs structurally hashed AIG (strash). aig rewrites each node's local function into an AIG while keeping the original network topology ($nd=9$, reported $aig=12$, $lev=3$). strash builds a single structurally hashed AIG; duplicate AND nodes are shared ($and=12$, $lev=4$). So strash is the global hashed DAG; aig is local AIG functions on the old netlist.

Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 9 logic nodes and 0 latches.



Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 12 logic nodes and 0 latches.

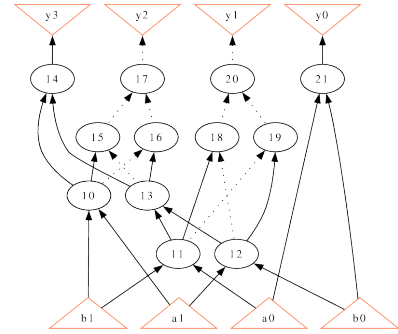
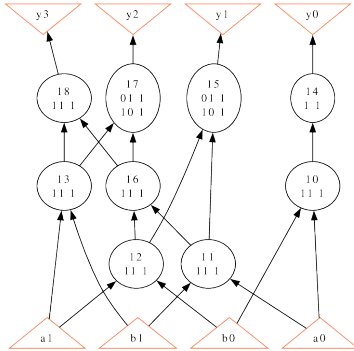


Figure 4: Left: after aig. Right: after strash.

BDD (bdd) vs collapsed BDD (collapse). bdd attaches a local BDD to each logic node ($nd=9$, $bdd=16$, $lev=3$). collapse builds global BDDs for the POs and replaces the network by those functions ($nd=4$, $bdd=14$, $lev=1$). Collapsed BDDs remove intermediate nodes and expose the full I/O function.

Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 9 logic nodes and 0 latches.



Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 16:58:10 2026.

The network contains 4 logic nodes and 0 latches.

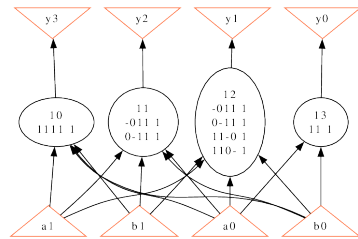


Figure 5: Left: after bdd (show). Right: after collapse (show bdd -g).

3(b) Strashed AIG $\rightarrow$ SOP logic network

Command sequence used:

```
read mul.blif
strash
logic
sop
print_stats
print_factor
```

logic converts the AIG back to a logic network; **sop** expresses each node in sum-of-products. Result: *nd*=12, *cube*=12, *lev*=4, with factored forms printable by **print_factor**.